

Exercitii practice pe grafuri desenate

Pasul 2 - tipuri de cautare pentru KR

BFS, DFS, DLS, IDS, UCS, Greedy, A*, euristici, consistenta, cautare bidirectionala

Scopul documentului: sa inveti cum se rezolva la examen exercitiile cu grafuri, fara cod. La fiecare exemplu urmarim frontiera pas cu pas, pentru ca exact asta se cere de obicei la subiectele de cautare.

Fara cod - doar pasi, grafuri si explicatii

Contents

1	Regula generala pentru orice cautare pe graf	2
2	Exemplul 1 - BFS si DFS pe acelasi graf	2
2.1	Graf	2
2.2	BFS - cautare in latime	2
2.3	DFS - cautare in adancime	3
3	Exemplul 2 - DFS poate gasi o solutie mai proasta decat BFS	4
3.1	Graf	4
3.2	BFS	4
3.3	DFS	4
4	Exemplul 3 - DLS si IDS	4
4.1	Graf	4
4.2	DLS cu limita $l = 2$	5
4.3	DLS cu limita $l = 3$	5
4.4	IDS	5
5	Exemplul 4 - UCS pe graf ponderat	6
5.1	Graf	6
5.2	Rezolvare	6
6	Exemplul 5 - Greedy poate fi pacalit	6
6.1	Graf cu euristici	7
6.2	Greedy	7
7	Exemplul 6 - A* repara problema lui Greedy	8
7.1	A*	8
8	Exemplul 7 - Verificarea admisibilitatii	8
8.1	Graf	8
8.2	Calculam costurile reale minime $h^*(n)$	8
8.3	Concluzie	9
9	Exemplul 8 - Euristica neadmisibila	9
10	Exemplul 9 - Consistenta euristicii	10
10.1	Graf	10
10.2	Verificare	10
11	Exemplul 10 - Cautare bidirectionala	10
11.1	Graf neponderat	10
11.2	Rezolvare	10
12	Exemplul 11 - Stari repetate: Tree Search vs Graph Search	12
12.1	Graf	12
12.2	Tree Search	12
12.3	Graph Search	12
13	Exemplul 12 - A* cu stare repetata si euristica consistenta	12
13.1	Graf	12
13.2	A* Graph Search	12

14 Subiecte de incercat singura	14
14.1 Subiectul 1 - BFS si DFS	14
14.2 Subiectul 2 - UCS	14
14.3 Subiectul 3 - Greedy si A*	14
14.4 Subiectul 4 - admisibilitate si consistenta	15
15 Raspunsuri scurte la subiectele de incercat	16
16 Fisa ultra-rapida	17

1 Regula generala pentru orice cautare pe graf

Inainte sa aplici un algoritm, noteaza mereu:

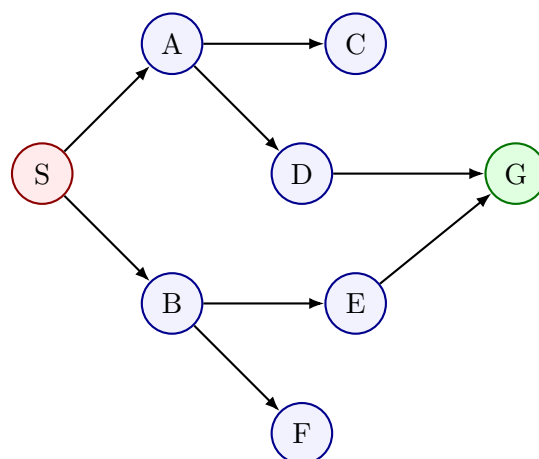
1. nodul de start S ;
2. nodul sau nodurile scop G ;
3. ordinea succesorilor, de obicei lexicografica: $A, B, C, \dots$;
4. daca muchiile au costuri sau nu;
5. daca exista euristica $h(n)$;
6. daca lucrezi cu arbore de cautare sau graf de cautare.

Algoritm	Alege urmatorul nod dupa	Ce scrii in examen
BFS	adancime minima, FIFO	frontiera pe niveluri
DFS	adancime maxima, LIFO	prima ramura cat mai jos, apoi backtracking
DLS	DFS cu limita l	DFS, dar nu cobori sub nivelul l
IDS	DLS repetat cu $l = 0, 1, 2, \dots$	arati iteratiile pana gasesti scopul
UCS	cost $g(n)$ minim	ordonezi frontiera dupa costul drumului
Greedy	euristica $h(n)$ minima	ignori costul deja parcurs
A*	$f(n) = g(n) + h(n)$ minim	calculezi g, h, f pentru fiecare drum
Bidirectionala	doua cautari, din start si din scop	opresti cand frontierele se intersecteaza

Observatie: In exemplele de mai jos, daca doua noduri au aceeasi prioritate, alegem lexicografic. La examen scrie explicit aceasta conventie daca nu este data.

2 Exemplul 1 - BFS si DFS pe acelasi graf

2.1 Graf



Ordinea succesorilor este lexicografica. Start: S . Scop: G .

2.2 BFS - cautare in latime

Rezolvare pas cu pas. BFS foloseste coada FIFO si exploreaza pe niveluri.

Pas	Scot	Frontiera dupa expandare	Observatie
0	-	$[S]$	pornim

1	S	$[S - A, S - B]$	nivelul 1
2	$S - A$	$[S - B, S - A - C, S - A - D]$	continuuam FIFO
3	$S - B$	$[S - A - C, S - A - D, S - B - E, S - B - F]$	final nivel 1
4	$S - A - C$	$[S - A - D, S - B - E, S - B - F]$	C nu are scop
5	$S - A - D$	$[S - B - E, S - B - F, S - A - D - G]$	generam G
6	$S - B - E$	$[S - B - F, S - A - D - G, S - B - E - G]$	mai exista drum la G
7	$S - B - F$	$[S - A - D - G, S - B - E - G]$	fara scop
8	$S - A - D - G$	-	scoatem scopul

Solutie BFS:

$$S - A - D - G.$$

Observatie: BFS gaseste solutia cu numar minim de muchii. Daca toate costurile sunt 1, atunci este si optima dupa cost.

2.3 DFS - cautare in adancime

Rezolvare pas cu pas. DFS merge cat mai adanc pe prima ramura disponibila. Folosim ordinea lexicografica.

Pas	Scot	Frontiera	Observatie
0	-	$[S]$	pornim
1	S	$[S - A, S - B]$	alegem A
2	$S - A$	$[S - A - C, S - A - D, S - B]$	primul coboram pe A
3	$S - A - C$	$[S - A - D, S - B]$	C nu are succesori
4	$S - A - D$	$[S - A - D - G, S - B]$	mergem spre G
5	$S - A - D - G$	-	scop gasit

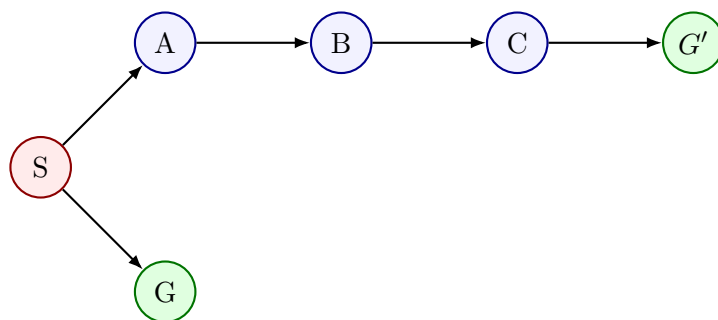
Solutie DFS:

$$S - A - D - G.$$

Observatie: Pe acest graf, BFS si DFS au gasit aceeasi solutie. Asta nu se intampla mereu. DFS poate gasi o solutie mai lunga, pentru ca nu cauta pe niveluri.

3 Exemplul 2 - DFS poate gasi o solutie mai proasta decat BFS

3.1 Graf



Start: S . Orice nod scop este acceptat: G sau G' . Ordinea succesorilor: A inainte de G .

3.2 BFS

Rezolvare pas cu pas. BFS vede mai intai nodurile de la adancimea 1:

$$[S] \rightarrow [S - A, S - G].$$

Urmatorul dupa $S - A$ este $S - G$, care este scop. Solutia BFS este:

$$S - G.$$

Are lungime 1.

3.3 DFS

Rezolvare pas cu pas. DFS merge pe prima ramura:

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G'.$$

Solutia DFS este:

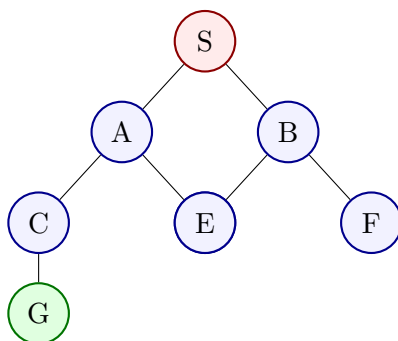
$$S - A - B - C - G'.$$

Are lungime 4.

Observatie: Aici vezi clar diferenta: BFS gaseste scopul cel mai apropiat ca numar de muchii, DFS poate intra pe prima ramura si sa gaseasca o solutie mai lunga.

4 Exemplul 3 - DLS si IDS

4.1 Graf



Scopul G este la adancimea 3.

4.2 DLS cu limita $l = 2$

Rezolvare pas cu pas. DLS se comporta ca DFS, dar nu trece de nivelul 2.

Niveluri:

$$S : 0, \quad A, B : 1, \quad C, D, E, F : 2, \quad G : 3.$$

Cu $l = 2$, algoritmul nu are voie sa ajunga la G .

Concluzie:

DLS cu $l = 2$ esueaza, desi solutia exista.

4.3 DLS cu limita $l = 3$

Rezolvare pas cu pas. Acum poate cobori pana la G :

$$S \rightarrow A \rightarrow C \rightarrow G.$$

Concluzie:

DLS cu $l = 3$ gaseste solutia.

4.4 IDS

Rezolvare pas cu pas. IDS ruleaza DLS repetat:

Iteratie	Ce viziteaza	Rezultat
$l = 0$	S	nu gaseste
$l = 1$	S, A, B	nu gaseste
$l = 2$	S, A, C, D, B, E, F	nu gaseste
$l = 3$	S, A, C, G	gaseste

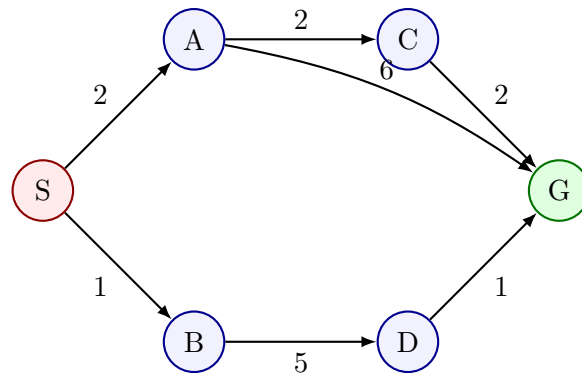
Solutie IDS:

$$S - A - C - G.$$

Observatie: IDS are avantajul BFS: gaseste solutia cea mai superficiala. Dar are memoria mica a DFS. Dezavantaj: reia cautarea de la radacina pentru fiecare limita.

5 Exemplul 4 - UCS pe graf ponderat

5.1 Graf



Start: S . Scop: G . UCS alege drumul cu $g(n)$ minim.

5.2 Rezolvare

Pas	Scot	Frontiera ordonata dupa cost g	Obs.
0	-	$S(0)$	start
1	$S(0)$	$S - B(1)$, $S - A(2)$	B e mai ieftin
2	$S - B(1)$	$S - A(2)$, $S - B - D(6)$	adaug D
3	$S - A(2)$	$S - A - C(4)$, $S - B - D(6)$, $S - A - G(8)$	C devine cel mai ieftin
4	$S - A - C(4)$	$S - B - D(6)$, $S - A - C - G(6)$, $S - A - G(8)$	am generat G cu cost 6
5	$S - B - D(6)$	$S - A - C - G(6)$, $S - B - D - G(7)$, $S - A - G(8)$	egalitate, lexicografic
6	$S - A - C - G(6)$	-	scop extras

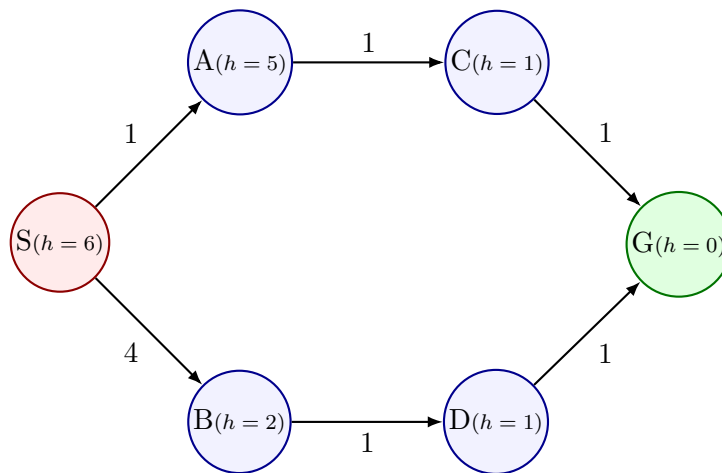
Solutie UCS:

$$S - A - C - G, \quad cost = 6.$$

Observatie: UCS nu se opreste cand genereaza pentru prima data scopul, ci cand scoate scopul din frontiera. Motiv: poate exista un drum spre scop generat mai tarziu, dar mai ieftin.

6 Exemplul 5 - Greedy poate fi pacalit

6.1 Graf cu euristici



6.2 Greedy

Greedy foloseste doar:

$$f(n) = h(n).$$

Rezolvare pas cu pas.

Pas	Scot	Frontiera dupa h	Obs.
0	-	$S(h=6)$	start
1	S	$B(h=2), A(h=5)$	Greedy alege B
2	B	$D(h=1), A(h=5)$	continua spre D
3	D	$G(h=0), A(h=5)$	G e minim
4	G	-	scop

Solutie Greedy:

$$S - B - D - G, \quad cost = 4 + 1 + 1 = 6.$$

Dar exista drumul:

$$S - A - C - G, \quad cost = 1 + 1 + 1 = 3.$$

Observatie: Greedy este atras de nodul cu euristica mica, dar ignora costul deja platit. De aceea poate fi neoptimal.

7 Exemplul 6 - A* repara problema lui Greedy

Folosim acelasi graf ca in exemplul anterior.

7.1 A*

A* foloseste:

$$f(n) = g(n) + h(n).$$

Rezolvare pas cu pas.

Pas	Scot	Frontiera ordonata dupa $f = g + h$	Obs.
0	-	$S : g = 0, h = 6, f = 6$	start
1	S	$A : g = 1, h = 5, f = 6$; $B : g = 4, h = 2, f = 6$	egalitate, aleg A
2	A	$C : g = 2, h = 1, f = 3$; $B : g = 4, h = 2, f = 6$	C e cel mai bun
3	C	$G : g = 3, h = 0, f = 3$; $B : g = 4, h = 2, f = 6$	scop cu cost 3
4	G	-	scop extras

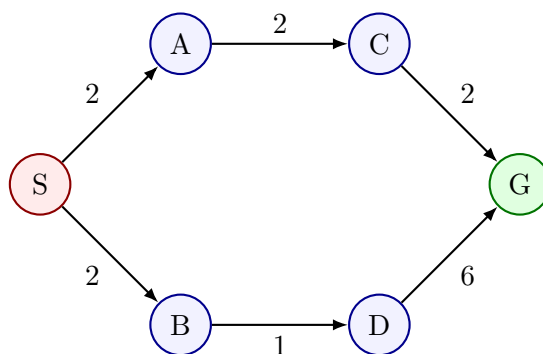
Solutie A*:

$$S - A - C - G, \quad cost = 3.$$

Observatie: A* nu se uita doar la distanta estimata pana la scop, ci si la costul parcurs deja. Din acest motiv poate evita o ramura aparent buna, dar scumpa.

8 Exemplul 7 - Verificarea admisibilitatii

8.1 Graf



Se da euristica:

$$h(S) = 5, \quad h(A) = 4, \quad h(B) = 4, \quad h(C) = 2, \quad h(D) = 6, \quad h(G) = 0.$$

8.2 Calculam costurile reale minime $h^*(n)$

Nod	Cel mai scurt drum pana la G	$h^*(n)$	$h(n)$
G	G	0	0
C	$C - G$	2	2
D	$D - G$	6	6
A	$A - C - G$	4	4

B	$B - D - G$	7	4
S	$S - A - C - G$	6	5

8.3 Concluzie

Pentru fiecare nod avem:

$$h(n) \leq h^*(n).$$

Deci euristica este admisibila.

Observatie: Ca sa demonstrezi ca o euristica este admisibila, verifici toate nodurile relevante. Ca sa demonstrezi ca NU este admisibila, ajunge un singur contraexemplu.

9 Exemplul 8 - Euristica neadmisibila

Folosim acelasi graf, dar schimbam:

$$h(C) = 5.$$

Costul real minim din C pana la G este:

$$h^*(C) = 2.$$

Dar:

$$h(C) = 5 > 2.$$

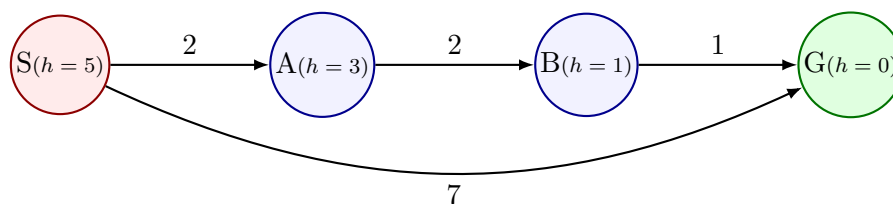
Concluzie:

euristica nu este admisibila.

Formulare de examen: Euristica nu este admisibila deoarece supraestimeaza costul real pentru nodul C : avem $h(C) = 5$, dar costul real minim pana la scop este $h^*(C) = 2$.

10 Exemplul 9 - Consistenta euristicii

10.1 Graf



O euristica este consistenta daca pentru fiecare muchie (u, v) :

$$h(u) \leq c(u, v) + h(v).$$

Pentru grafuri neorientate se foloseste deseori si forma:

$$|h(u) - h(v)| \leq c(u, v).$$

10.2 Verificare

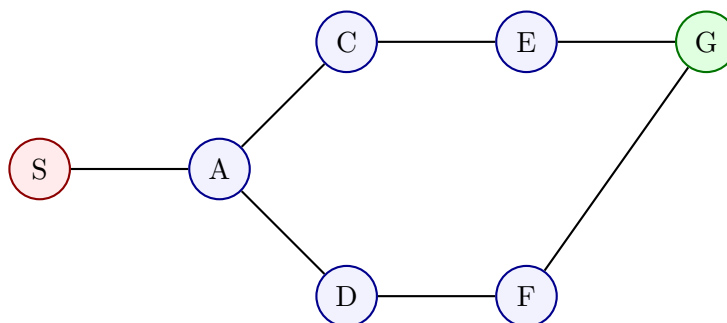
Muchie	Cost	Verificare	Rezultat
$S - A$	2	$ 5 - 3 = 2 \leq 2$	ok
$A - B$	2	$ 3 - 1 = 2 \leq 2$	ok
$B - G$	1	$ 1 - 0 = 1 \leq 1$	ok
$S - G$	7	$ 5 - 0 = 5 \leq 7$	ok

Concluzie: euristica este consistenta.

Observatie: Orice euristica consistenta este si admisibila. Pentru A* Graph Search, consistenta este conditia importanta de optimalitate.

11 Exemplul 10 - Cautare bidirectionala

11.1 Graf neponderat



11.2 Rezolvare

Cautarea bidirectionala porneste simultan:

- din S spre G ;
- din G spre S .

Nivel	Dinspre S	Dinspre G	Intersectie
0	$\{S\}$	$\{G\}$	nu

1	$\{A\}$	$\{E, F\}$	nu
2	$\{C, D\}$	$\{C, D\}$	da

Intersectia se face in C sau D . Un drum posibil:

$$S - A - C - E - G.$$

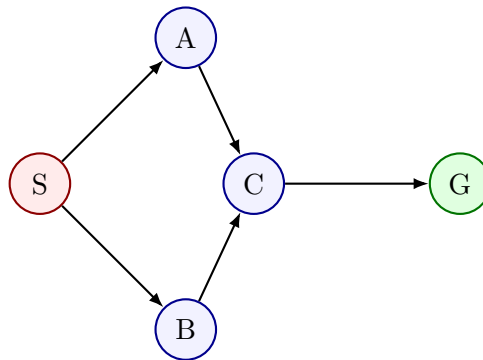
Alt drum posibil:

$$S - A - D - F - G.$$

Observatie: Cautarea bidirectionala este buna cand stim clar si startul, si scopul, iar putem genera predecesori dinspre scop.

12 Exemplul 11 - Stari repetate: Tree Search vs Graph Search

12.1 Graf



Starea C poate fi atinsa pe doua drumuri:

$$S - A - C \quad \text{si} \quad S - B - C.$$

12.2 Tree Search

Tree Search trateaza fiecare drum ca nod diferit in arbore. Deci poate avea in frontiera:

$$S - A - C \quad \text{si} \quad S - B - C.$$

Asta poate duce la munca duplicata.

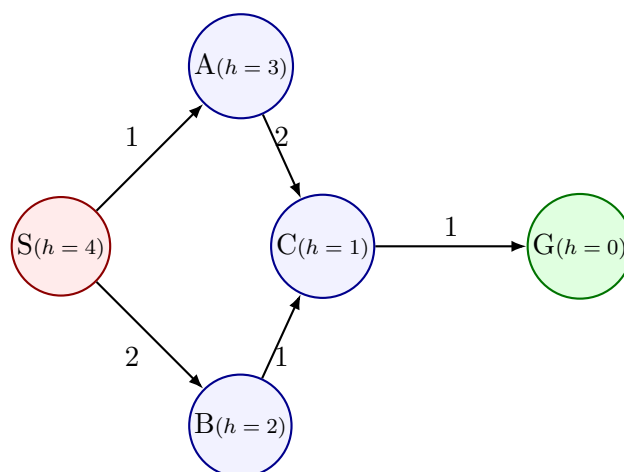
12.3 Graph Search

Graph Search tine o multime **Closed** cu starile deja expandate. Dupa ce a expandat C prima data, daca il intalneste din nou, il ignora.

Observatie: Graph Search reduce repetarile, dar la A^* trebuie avut grija: daca euristica nu este consistenta, exista cazuri in care ignorarea unei stari deja explorate poate elimina un drum mai bun.

13 Exemplul 12 - A^* cu stare repetata si euristica consistenta

13.1 Graf



13.2 A^* Graph Search

Pas	Scot	Open dupa $f = g + h$	Closed
0	-	$S : f = 4$	$\emptyset$
1	S	$A : g = 1, h = 3, f = 4; B : g = 2, h = 2, f = 4$	$\{S\}$
2	A	$B : f = 4; C : g = 3, h = 1, f = 4$	$\{S, A\}$
3	B	$C : g = 3, h = 1, f = 4; C : g = 3, h = 1, f = 4$	$\{S, A, B\}$
4	C	$G : g = 4, h = 0, f = 4$	$\{S, A, B, C\}$
5	G	-	scop

Solutie:

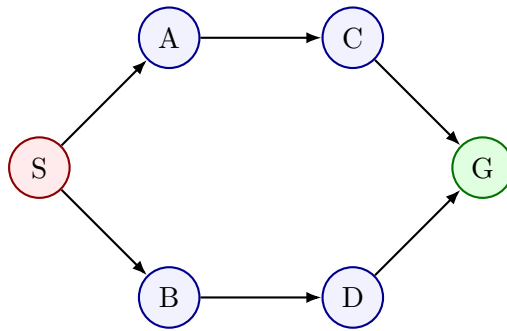
$$S - A - C - G \quad \text{sau} \quad S - B - C - G,$$

ambele cu cost 4.

Observatie: Daca gasim acelasi nod cu acelasi cost, nu conteaza care drum ramane. Daca un drum are cost mai mic, trebuie pastrat drumul cu g minim.

14 Subiecte de incercat singura

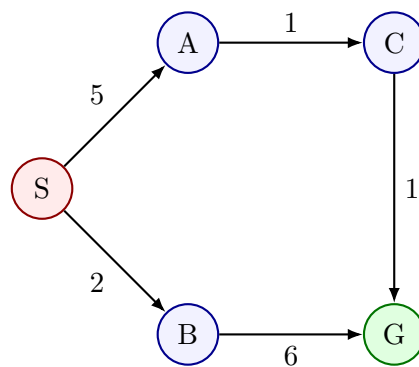
14.1 Subiectul 1 - BFS si DFS



Cerinte:

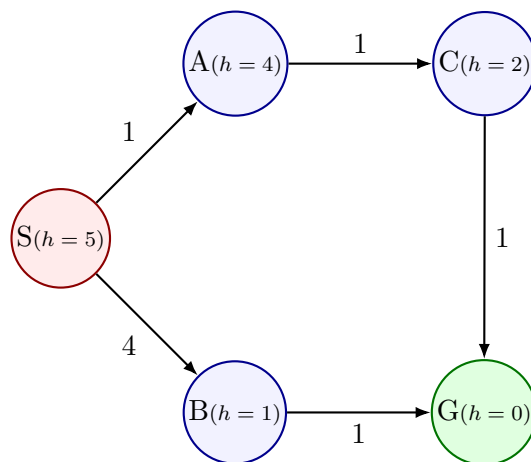
1. Aplicati BFS.
2. Aplicati DFS.
3. Spuneti daca solutiile sunt garantat optime.

14.2 Subiectul 2 - UCS



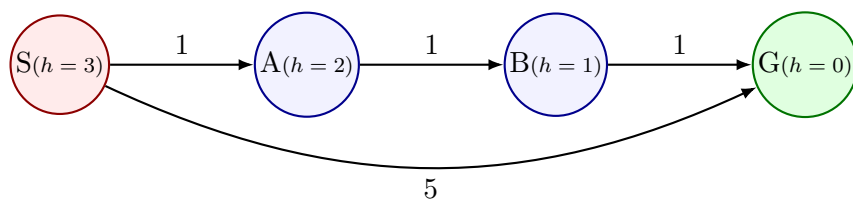
Cerinta: Aplicati UCS si dati drumul optim.

14.3 Subiectul 3 - Greedy si A*



Cerinte:

1. Aplicati Greedy.
2. Aplicati A*.
3. Comparati costurile obtinute.

14.4 Subiectul 4 - admisibilitate si consistenta

Cerinte:

1. Verificati daca euristica este admisibila.
2. Verificati daca euristica este consistenta.

Raspunsuri scurte la subiectele de incercat

Raspuns subiect 1

BFS:

$$S \rightarrow A, B \rightarrow C, D \rightarrow G.$$

Cu verificare la scoatere, un drum posibil este:

$$S - A - C - G.$$

DFS, cu ordine lexicografica:

$$S - A - C - G.$$

BFS este optim pentru costuri egale. DFS nu este garantat optim in general.

Raspuns subiect 2

UCS:

$$\begin{aligned} &S(0) \\ &S - B(2), S - A(5) \\ &S - A(5), S - B - G(8) \\ &S - A - C(6), S - B - G(8) \\ &S - A - C - G(7), S - B - G(8) \end{aligned}$$

Solutie:

$$S - A - C - G, \quad cost = 7.$$

Raspuns subiect 3

Greedy:

$$S \rightarrow B \rightarrow G, \quad cost = 4 + 1 = 5.$$

A*:

$$S - A : f = 1 + 4 = 5, \quad S - B : f = 4 + 1 = 5.$$

La egalitate alegem lexicografic A:

$$\begin{aligned} &S - A - C : g = 2, h = 2, f = 4, \\ &S - A - C - G : g = 3, h = 0, f = 3. \end{aligned}$$

Solutie A*:

$$S - A - C - G, \quad cost = 3.$$

Raspuns subiect 4

Costuri reale:

$$h^*(G) = 0, \quad h^*(B) = 1, \quad h^*(A) = 2, \quad h^*(S) = 3.$$

Euristica este admisibila, pentru ca $h(n) = h^*(n)$ pentru fiecare nod.

Consistentia:

$$\begin{aligned} |h(S) - h(A)| &= 1 \leq 1, \\ |h(A) - h(B)| &= 1 \leq 1, \\ |h(B) - h(G)| &= 1 \leq 1, \\ |h(S) - h(G)| &= 3 \leq 5. \end{aligned}$$

Deci este consistenta.

16 Fisa ultra-rapida

Algoritm	Cum aleg nodul	Propozitia de examen
BFS	cel mai vechi nod din frontiera	cauta pe niveluri
DFS	cel mai recent nod din frontiera	merge pe o ramura pana jos
DLS	DFS pana la limita l	poate rata solutia daca l e prea mic
IDS	DLS pentru $l = 0, 1, 2, \dots$	combina memoria mica DFS cu optimalitatea BFS pentru costuri egale
UCS	cost g minim	optim pentru costuri pozitive
Greedy	h minim	rapid, dar neoptimal
A*	$g + h$ minim	optim cu euristica admisi-bila/consistenta, in functie de varianta
Bidirectionala	doua cautari	se opreste la intersectia frontierelor